

WEEK 1 / Session 1 :

Introduction to Functional Programming Concepts:

1. Java Lambda Expressions

Java Code:

```
import java.io.*;
```

```
import java.util.*;
```

```
interface PerformOperation {
```

```
    boolean check(int a);
```

```
}
```

```
class MyMath {
```

```
    public static boolean checker(PerformOperation p, int num) {
```

```
        return p.check(num);
```

```
}
```

```
    public static PerformOperation isOdd() {
```

```
        return (int a) -> a % 2 != 0;
```

```
}
```

```
    public static PerformOperation isPrime() {
```

```
        return (int a) -> {
```

```
            if (a < 2) {
```

```
                return false;
```

```
            }
```

```
        for (int i = 2; i * i <= a; i++) {  
            if (a % i == 0) {  
                return false;  
            }  
        }  
  
        return true;  
    };  
}
```

```
public static PerformOperation isPalindrome() {  
    return (int a) -> {  
        int original = a;  
        int reverse = 0;  
  
        while (a != 0) {  
            int digit = a % 10;  
            reverse = reverse * 10 + digit;  
            a = a / 10;  
        }  
  
        return original == reverse;  
    };  
}
```

```
public class Solution {  
    VTU29399
```

```
public static void main(String[] args) throws IOException {

    MyMath ob = new MyMath();

    BufferedReader br =
        new BufferedReader(new InputStreamReader(System.in));

    int T = Integer.parseInt(br.readLine());

    PerformOperation op;
    boolean ret = false;
    String ans = null;

    while (T-- > 0) {

        String s = br.readLine().trim();
        StringTokenizer st = new StringTokenizer(s);

        int ch = Integer.parseInt(st.nextToken());
        int num = Integer.parseInt(st.nextToken());

        if (ch == 1) {

            op = ob.isOdd();
            ret = ob.checker(op, num);
            ans = (ret) ? "ODD" : "EVEN";
```

```
    } else if (ch == 2) {  
  
        op = ob.isPrime();  
        ret = ob.checker(op, num);  
        ans = (ret) ? "PRIME" : "COMPOSITE";  
  
    } else if (ch == 3) {  
  
        op = ob.isPalindrome();  
        ret = ob.checker(op, num);  
        ans = (ret) ? "PALINDROME" : "NOT PALINDROME";  
    }  
    System.out.println(ans);  
}  
}}
```

Output:

EVEN

PRIME

PALINDROME

ODD

COMPOSITE

2. Java Comparator:

Java code:

```
import java.util.*;

class Player {
    String name;
    int score;

    Player(String name, int score) {
        this.name = name;
        this.score = score;
    }
}

class Checker implements Comparator<Player> {
    public int compare(Player a, Player b) {
        if (a.score != b.score) {
            return b.score - a.score;
        }

        return a.name.compareTo(b.name);
    }
}

public class Solution {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        int n = scan.nextInt();

        Player[] players = new Player[n];

        for (int i = 0; i < n; i++) {
            players[i] = new Player(scan.next(), scan.nextInt());
        }

        Checker checker = new Checker();

        Arrays.sort(players, checker);
    }
}
```

```
        for (Player player : players) {  
            System.out.println(player.name + " " + player.score);  
        }  
  
        scan.close();  
    }  
}
```

Output:

```
aleksa 150  
amy 100  
david 100  
aakansha 75  
heraldo 50
```

Session 2 :

Lambda Expressions

1.Java Sort:

Java code :

```
import java.util.*;
```

```
class Student {
```

```
    private int id;
```

```
    private String name;
```

```
    private double cgpa;
```

```
    public Student(int id, String name, double cgpa) {
```

```
        this.id = id;
```

```
        this.name = name;
```

```
        this.cgpa = cgpa;
```

```
    }
```

```
    public int getId() {
```

```
        return id;
```

```
    }
```

```
    public String getName() {
```

```
        return name;
```

```
    }
```

```
    public double getCgpa() {
```

```
        return cgpa;
```

VTU29399

```
}  
}
```

```
public class Solution {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        int n = sc.nextInt();
```

```
        List<Student> students = new ArrayList<>();
```

```
        for (int i = 0; i < n; i++) {
```

```
            int id = sc.nextInt();
```

```
            String name = sc.next();
```

```
            double cgpa = sc.nextDouble();
```

```
            students.add(new Student(id, name, cgpa));
```

```
        }
```

```
        Collections.sort(students, new Comparator<Student>() {
```

```
            public int compare(Student a, Student b) {
```

```
                if (a.getCgpa() != b.getCgpa()) {
```

```
                    return Double.compare(b.getCgpa(), a.getCgpa());
```

```
VTU29399
```

```

    }

    int nameCompare = a.getName().compareTo(b.getName());

    if (nameCompare != 0) {
        return nameCompare;
    }

    return Integer.compare(a.getId(), b.getId());
}

});

for (Student student : students) {
    System.out.println(student.getName());
}

sc.close();
}}

```

Output:

Ashis
 Fahim
 Samara
 Samiha
 Rumpa

Session 3 :

Anonymous Functions & Functional Interfaces:

1. Sorting: Comparator :

VTU29399

Java code:

```
import java.util.*;

class Player {
    String name;
    int score;

    Player(String name, int score) {
        this.name = name;
        this.score = score;
    }
}

class Checker implements Comparator<Player> {
    public int compare(Player a, Player b) {
        if (a.score != b.score) {
            return b.score - a.score;
        }
        return a.name.compareTo(b.name);
    }
}

public class Solution {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        int n = scan.nextInt();

        Player[] player = new Player[n];
        Checker checker = new Checker();
        for (int i = 0; i < n; i++) {
            player[i] = new Player(scan.next(), scan.nextInt());
        }
    }
}
```

VTU29399

```
    }  
scan.close();  
Arrays.sort(player, checker);  
    for (int i = 0; i < player.length; i++) {  
        System.out.printf("%s %s\n", player[i].name, player[i].score);  
    }  
}  
}
```

Output:

```
aleksa 150  
amy 100  
david 100  
aakansha 75  
heraldo 50
```

Session 4:

Higher-Order Functions

1. Running Sum of 1D Array –

Java code:

```
class Solution {  
    public int[] runningSum(int[] nums) {  
  
        int sum = 0;  
  
        for (int i = 0; i < nums.length; i++) {  
            sum = sum + nums[i];  
            nums[i] = sum;  
        }  
  
        return nums;  
    }  
}
```

Output:

[1,3,6,10]

2. Richest Customer Wealth –

Java code:

```
class Solution {  
    public int maximumWealth(int[][] accounts) {  
  
        int maxWealth = 0;  
  
        for (int i = 0; i < accounts.length; i++) {  
  
            int wealth = 0;  
  
            for (int j = 0; j < accounts[i].length; j++) {  
                wealth = wealth + accounts[i][j];  
            }  
  
            if (wealth > maxWealth) {  
                maxWealth = wealth;  
            }  
        }  
  
        return maxWealth;  
    }  
}
```

Output/Input:

```
[[1,2,3],[3,2,1]]
```

```
6
```

Session 5

Higher-Order Functions

1. Squares of a Sorted Array –

Java code:

```
class Solution {  
    public int[] sortedSquares(int[] nums) {  
  
        int n = nums.length;  
        int[] result = new int[n];  
  
        int left = 0;  
        int right = n - 1;  
  
        for (int i = n - 1; i >= 0; i--) {  
  
            int leftSquare = nums[left] * nums[left];  
            int rightSquare = nums[right] * nums[right];  
  
            if (leftSquare > rightSquare) {  
                result[i] = leftSquare;  
                left++;  
            } else {  
                result[i] = rightSquare;  
                right--;  
            }  
        }  
    }  
}
```

```
return result;
```

```
}  
}
```

Output:

[4,9,9,49,121]

3. Find Pivot Index –

Java code:

```
class Solution {  
    public int pivotIndex(int[] nums) {  
  
        int totalSum = 0;  
  
        for (int num : nums) {  
            totalSum += num;  
        }  
  
        int leftSum = 0;  
  
        for (int i = 0; i < nums.length; i++) {  
  
            int rightSum = totalSum - leftSum - nums[i];  
  
            if (leftSum == rightSum) {  
                return i;  
            }  
  
            leftSum += nums[i];  
        }  
  
        return -1;  
    }  
}
```

Output/Input:

nums =

[1,7,3,6,5,6]

3